/* ClingSDKDemo Guide

Created by zhusanbao on 2024/8/21.

*/

Documentation for the ClingSDKDemo

Online doc (https://testflight.hicling.com/sdkdemodoc.html)

Some individual developer code has been removed to make the code easier to understand. This demo demonstrates the functionality of the Cling Commercial SDK for searching, activating, connecting, disconnecting, data syncing, and configuring Cling devices. I hope this will be helpful for you.

The minimum supported iOS version for the Cling SDK is iOS 12.0.

Before starting, please add the `ClingSDK.framework` to your project. Then, include the following keys in the `info.plist` file:

- `<key>NSBluetoothAlwaysUsageDescription</key>` `<string>xxx</string>` (Explanation for requesting Bluetooth permission)
- `<key>NSAllowsArbitraryLoads</key>` `<true/>`

This demo has already completed these additions and can be directly run on a physical device.

Overview

This documentation provides an explanation of the `ViewController` class, which manages various device interactions such as scanning, registering, syncing data, and more using a Bluetooth-enabled Cling device. The class utilizes the Cling SDK to handle device connectivity and synchronization. The class is divided into several sections, each with specific functionality.

Basic Workflow

1. **Initialize the ViewController**:
   - When the view is loaded, `viewDidLoad()` is called. This method registers the necessary elements and observers to enable communication with the Cling SDK.

2. **Device Operations**:
   - The class handles device scanning, registration, connection, and data synchronization with the help of the Cling SDK.
3. **Observer Notifications**:
   - Multiple observers are registered to listen for updates related to device state changes, data sync, firmware updates, and more.
4. **Actions on Device**:
   - Various actions are available to perform device operations such as setting reminders, weather configurations, push notifications, and device language.

Device Basic Flow

1. **Set the type of scanning device**:
   - ClingBLEModel.sharedInstance().setClingDeviceStyle(.GE)
2. **Scan for devices**:
   - ClingBLEModel.sharedInstance().startDiscoveryDevice()
3. **If the specified device is found, proceed to step 4; otherwise, continue scanning**:
4. **If the device is already activated, connect to the device; otherwise, activate and automatically connect to the device**:
   - `ClingBLEModel.sharedInstance().registerDevice("A9CF", bondUid: 1001, complete: {})` -- activate device
   - Update the locally cached user ID bound to the cling device. This method should be used when a cling device is already bound.To establish a connection with a bound device, call this method first, then call the tryConnectDevice method.
   - bid: The new binding ID. By default, the locally cached user ID is the bid passed in by the registerDevice method.
   - If the cling device is already activated and the locally cached bid does not match the user ID actually bound to the device,
   - you need to pass in the user ID bound to the device.
   - `ClingBLEModel.updateBondUid(1001, clingID: clingid)` & `ClingBLEModel.sharedInstance().tryConnectDevice()` -- connect to activated device
5. **Communicate with the device**

Detailed Breakdown of `viewDidLoad` Method

`viewDidLoad()`

- **Purpose**: register the Cling SDK, add observers for device notifications, and attempt to activate the last connected device.
  - **`registerCling()`**: Registers the Cling SDK using an app ID and secret.
  - **`addClingObserver()`**: Adds several observers to listen for various notifications from the Cling device (e.g., sync data, connection status).
  - **`activeLastDevice()`**: Attempts to connect the last registered device.

`registerCling()` Method

`registerCling()`

- **Purpose**: Registers the Cling SDK by providing the application ID, secret, and specifying whether the app is an enterprise application.

`addClingObserver()`

- **Purpose**: Adds observers to handle various notifications from the Cling device. These include data sync notifications, device discovery, connection status, GPS data, and more.

Actions for Device Operations

These methods are mapped to the various actions within the app's UI, allowing the user to interact with the device.

`actionScanDevice()`

- **Purpose**: Initiates scanning for available devices. If a device is found, it prepares for activation and registration.
- **Process**: The Cling device type is set and scanning begins for 10 seconds. After scanning, it automatically attempts registration.

`actionRegisterDevice()`

- **Purpose**: Registers the scanned device using its Bluetooth code and binds it to the app.

`actionDeregiestDevice()`

- **Purpose**: Deregisters the connected device, effectively "unbinding" it from the app.

`actionDeviceDisconnect()`

- **Purpose**: Disconnects the currently connected device, making it available for another connection.

Actions Related to Device Data and Settings

`actionSyncDeviceData()`

- **Purpose**: Syncs data from the connected device, such as minute-by-minute heart rate and activity.

`actionSetPushTypes()`

- **Purpose**: Configures the types of notifications the device will receive, such as calls, social media, and location alerts.

CLING_WATCH_NOTIFICATION list

- **CLING_WATCH_NOTIFICATION_CLOSE_ALL**: 关闭所有通知 (Close all notifications)
- **CLING_WATCH_NOTIFICATION_ID_INCOMING_CALL**: 来电 (Incoming call)
- **CLING_WATCH_NOTIFICATION_ID_MISSED_CALL**: 未接来电 (Missed call)
- **CLING_WATCH_NOTIFICATION_ID_VOICE_MAIL**: 语音邮件 (Voicemail)
- **CLING_WATCH_NOTIFICATION_ID_SOCIAL**: 社交 (Social)
- **CLING_WATCH_NOTIFICATION_ID_SCHEDULE**: 日程 (Schedule)
- **CLING_WATCH_NOTIFICATION_ID_EMAIL**: 电子邮件 (Email)
- **CLING_WATCH_NOTIFICATION_ID_NEWS**: 新闻 (News)
- **CLING_WATCH_NOTIFICATION_ID_HEALTH_FITNESS**: 健身健康 (Health & Fitness)
- **CLING_WATCH_NOTIFICATION_ID_BUSINESS_FINANCE**: 商务金融 (Business & Finance)
- **CLING_WATCH_NOTIFICATION_ID_LOCATION**: 位置 (Location)
- **CLING_WATCH_NOTIFICATION_ID_ENTERTAINMENT**: 娱乐 (Entertainment)

`actionSetReminder()`

- **Purpose**: Sets reminders on the device. The device can store up to 32 reminders. Sending an empty array removes all reminders.
- Field Description:
- **iReminderId**: 提醒 ID [1, 65535] (Reminder ID, range [1, 65535])
- **iReminderHour**: 小时 [0, 23] (Hour, range [0, 23])

- **iReminderMinute**: 分钟 [0, 59] (Minute, range [0, 59])
- **strName**: 提醒名 可选 长度不超过128, 目前暂不支持此属性配置 (Reminder name, optional, max length 128, currently this property is not supported for configuration)
- **weekday**: 星期 (Weekday for the reminder)

actionDeviceSetWeather()

- **Purpose**: Configures the weather information on the device for up to 5 future days.
- Field `ClingDeviceWeather` Description:
- **intWeatherTime**: 天气时间戳 (Weather timestamp)
- **style**: 天气 (Weather style)
- **fTempLow**: 最低温度 (Lowest temperature)
- **fTempHigh**: 最高温度 (Highest temperature)
- **aqi**: aqi空气质量 (AQI - Air Quality Index)
- Field `ClingDeviceWeatherStyle` Description:
- **ClingDeviceWeatherStyleSunny**: Sunny
- **ClingDeviceWeatherStyleCloudy**: Cloudy
- **ClingDeviceWeatherStyleRainy**: Rainy
- **ClingDeviceWeatherStyleSnowy**: Snowy

actionSetDeviceConfig()

- **Purpose**: Configures various device settings such as idle alerts.
- Field Description `ClingDeviceCfg`
- **bFlipWrist**: 翻转手腕 (Flip wrist)
- **bTouchEnable**: 触摸屏开关 (Touch screen enable switch)
- **bBloodP**: 血压开关 (Auto Blood Pressure Monitor switch)
- **bBloodO**: 血氧开关 (Auto SpO2 Monitor switch)
- **bHold**: 点击并按住屏幕 (Press and hold screen)
- **bTap**: 轻拍屏幕 (Tap screen)
- **uint8Hold**: 按住屏幕时长 有效值：1、3 单位：秒 默认：1 (Hold screen duration, valid values: 1, 3, unit: seconds, default: 1)
- **uint8ScreenOff**: 关闭时间 有效值：[2,15] 单位：秒 默认：5 (Screen off time, valid values: [2,15], unit: seconds, default: 5)
- **uint8HROff**: 心率监测关闭时间 有效值：[15,60] 单位：秒 默认：25 (Heart rate monitoring off time, valid values: [15,60], unit: seconds, default: 25)
- **uint8HRDay**: 测量间隔（白天）有效值：[0,120] 单位：分 默认：15 ClingLeap有效值可以设置为0,其他是[5,120] (Measurement interval (daytime), valid values: [0,120], unit: minutes, default: 15; ClingLeap valid values: 0, others: [5,120])
- **uint8HRNight**: 测量间隔（夜间）有效值：[5,120] 单位：分 默认：30 (Measurement interval (nighttime), valid values: [5,120], unit: minutes, default: 30)
- **uint8TempDay**: 测量间隔（白天）有效值：[5,120] 单位：分 默认：15 (Measurement interval (daytime), valid values: [5,120], unit: minutes, default: 15)

- **uint8TempNight**: 测量间隔（夜间）有效值：[5,120] 单位：分 默认：30 (Measurement interval (nighttime), valid values: [5,120], unit: minutes, default: 30)
- **bIdleAlert**: 是否打开久坐提醒 (Enable idle alert)
- **uint8IdleAlert**: 提醒时间间隔 有效值：[30,150] 单位：分 默认：30 (Idle alert interval, valid values: [30,150], unit: minutes, default: 30)
- **intIdleHourStart**: 一天中起始小时 (Start hour of the day)
- **intIdleHourEnd**: 一天中终止小时 (End hour of the day)
- **uint8SleepSensitivity**: 睡眠灵敏度 0:低 1:中 2:高 (Sleep sensitivity, 0: low, 1: medium, 2: high)
- **bWeekendAlarmDisabled**: 周末提醒是否有效 (Whether weekend alarms are enabled)
- **intStepSensitivity**: 计步灵敏度 2:低 1:中 0:高 (Step sensitivity, 2: low, 1: medium, 0: high)
- **intVocSampleRate**: voc采样率 2:低 1:中 0:高 voc手环设置采样率使用 (VOC sample rate, 2: low, 1: medium, 0: high, used for VOC wristband sampling rate setting)
- **intAlcoholSensitivity**: 酒精监测灵敏度 2:低 1:中 0:高 (Alcohol monitoring sensitivity, 2: low, 1: medium, 0: high)
- **strCityName**: 天气使用的城市名 (City name used for weather)
- **iHrbroadcast**: 心率广播是否开启 0：关闭 1：开启 (Heart rate broadcast enabled: 0: off, 1: on)
- **iTimeOut**: 报警超时 有效值：[30,2400] 单位：秒 默认：30 (Alarm timeout, valid values: [30,2400], unit: seconds, default: 30)

`actionSetDeviceLanguage()`

- **Purpose**: Sets the device language. Only supported for devices other than watches.
- Field Description `ClingDeviceLanguage`
- `ClingDeviceLanguageEnglish`: English language
- `ClingDeviceLanguageSimplified`: Simplified language
- `ClingDeviceLanguageTraditional`: Traditional language

`actionDeviceCheckFirmwareUpdate()`

- **Purpose**: Checks if the connected device has a firmware update available. If an update is found, it initiates the upgrade process.

`actionDeviceUpdateUserProfile()`

- **Purpose**: Updates the user profile on the device with details like height, weight, stride length, and daily goals.
- Field Description `ClingPeripheralUserProfileModel`
- `height_in_cm`: user height. //
- `weight_in_kg`: user weight //
- `stride_length_in_cm`: stride length, i.e., step length //
- `stride_length_run_in_cm`: running stride length //

- `stepRateForRunLength`: Running step frequency //
- `units_type`: 0 international or 1 English unit
- `nickname`: Welcome message at startup, maximum length of 16 bytes
- `clock_orientation`: Device time display orientation, PERIPHERAL_PROFILE_CLOCK_ORIENTATION_VERTICAL | PERIPHERAL_PROFILE_CLOCK_ORIENTATION_HORIZONTAL
- `sleep_alarm_day_of_week`: Sleep alarm reminder time, CLING_DEVICE_REMINDER_WEEKDAY
- `bed_hr`: Bedtime hour
- `bed_min`: Bedtime minute
- `wakeup_hr`: Wake-up hour
- `wakeup_min`: Wake-up minute
- `screen_display_option`: Screen display option, see PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION related macros
- `touch_virbration`: Whether screen touch vibration is enabled, see PERIPHERAL_PROFILE_TOUCH_VIRBRATION related macros
- `daily_goal`: Daily kilometer goal in KMs
- `training_display_option`: Training display page, see PERIPHERAL_PROFILE_TRAINING_DISPLAY_OPTION related macros
- `age`: User's age
- `sex`: User's sex
- `stride_length_run_indoor_in_cm`: Indoor running stride length, such as treadmill running
- `hr_alarm_rate`: Heart rate zone alarm value, using PERIPHERAL_PROFILE_HEART_ALARM_ENABLE to enable HR alarm, default value is PERIPHERAL_PROFILE_HEART_ALARM_DEFAULT_VALUE
- `pace_alarm_zone`: Pace alarm value, using PERIPHERAL_PROFILE_PACE_ALARM_ENABLE to enable pace alarm, default value is PERIPHERAL_PROFILE_PACE_ALARM_DEFAULT_VALUE
- `caloryDisplayType`: Calorie display type: using CALORIES_DISPLAY_TYPE_ALL or CALORIES_DISPLAY_TYPE_ACTIVE
- `iHealthInfoLevel`: Health challenge level 0: Beginner 1: Basic 2: Active 3: Professional
- `bCalAlarm`: Whether calorie alarm is enabled, default is off

- `iCAlarmValue`: Calorie alarm value, default 1300 [1000, 5000]
- `bStepAlarm`: Whether step count alarm is enabled, default is on
- `iSAlarmValue`: Step count alarm value, default 10000 [10000, 30000]
- `iTempAlarmValue`: Temperature alarm value, default 373 (37.3*10)
- `iTempLowAlarmValue`: Low temperature alarm value, default 273 (27.3*10)
- Field Description `CLING_DEVICE_REMINDER_WEEKDAY`
- `CLING_DEVICE_REMINDER_WEEKDAY_MONDAY`: // Monday
- `CLING_DEVICE_REMINDER_WEEKDAY_TUESDAY`: // Tuesday
- `CLING_DEVICE_REMINDER_WEEKDAY_WEDNESDAY`: // Wednesday
- `CLING_DEVICE_REMINDER_WEEKDAY_THURSDAY`: // Thursday
- `CLING_DEVICE_REMINDER_WEEKDAY_FRIDAY`: // Friday
- `CLING_DEVICE_REMINDER_WEEKDAY_SATURDAY`: // Saturday
- `CLING_DEVICE_REMINDER_WEEKDAY_SUNDAY`: // Sunday
- `CLING_DEVICE_REMINDER_WEEKDAY_ALL`: // All days

Field Names and Comments

- **PERIPHERAL_PROFILE_CLOCK_ORIENTATION_VERTICAL**: Vertical display
- **PERIPHERAL_PROFILE_CLOCK_ORIENTATION_HORIZONTAL**: Horizontal display
- **PERIPHERAL_PROFILE_TOUCH_VIRBRATION_OFF**: Touchscreen vibration off
- **PERIPHERAL_PROFILE_TOUCH_VIRBRATION_ON**: Touchscreen vibration on

Display Settings (Valid if the device supports the corresponding features)

- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_STEP**: Display step module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_DISTANCE**: Display distance module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_CALORIES**: Display calories module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_ACTIVE_TIME**: Display active time module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_HEART_RATE**: Display heart rate module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_WEATHER**: Display weather module

- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_PM2P5**: Display air quality module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_START_RUNNING**: Display start running module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_ANALYSIS**: Display analysis module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_SKIN_TEMP**: Display skin temperature module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_UV**: Display UV module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_BLOODPRESSURE**: Display blood pressure module
- **PERIPHERAL_PROFILE_SCREEN_DISPLAY_OPTION_DEFAULT_VALUE**: Default screen display modules

Training Page Display Options

- **PERIPHERAL_PROFILE_TRAINING_DISPLAY_OPTION_DISTANCE**: Display distance in training page
- **PERIPHERAL_PROFILE_TRAINING_DISPLAY_OPTION_TIME**: Display total time in training page
- **PERIPHERAL_PROFILE_TRAINING_DISPLAY_OPTION_PACE**: Display average pace in training page
- **PERIPHERAL_PROFILE_TRAINING_DISPLAY_OPTION_STRIDE**: Display average stride in training page
- **PERIPHERAL_PROFILE_TRAINING_DISPLAY_OPTION_CADENCE**: Display average cadence in training page
- **PERIPHERAL_PROFILE_TRAINING_DISPLAY_OPTION_HEART_RATE**: Display average heart rate in training page
- **PERIPHERAL_PROFILE_TRAINING_DISPLAY_OPTION_CALORIES**: Display calories burned in training page
- **PERIPHERAL_PROFILE_TRAINING_DISPLAY_OPTION_DEFAULT_VALUE**: Default configuration for training page

Alarm Options

- **PERIPHERAL_PROFILE_HEART_ALARM_ENABLE**: Whether heart rate alarm is enabled. Enable: hr_alarm_rate |= PERIPHERAL_PROFILE_HEART_ALARM_ENABLE; Disable: hr_alarm_rate &= ~PERIPHERAL_PROFILE_HEART_ALARM_ENABLE;
- **PERIPHERAL_PROFILE_HEART_ALARM_DEFAULT_VALUE**: Default heart rate alarm value
- **PERIPHERAL_PROFILE_PACE_ALARM_ENABLE**: Whether pace alarm is enabled. Enable: pace_alarm_zone |= PERIPHERAL_PROFILE_HEART_ALARM_ENABLE; Disable: pace_alarm_zone &= ~PERIPHERAL_PROFILE_HEART_ALARM_ENABLE;
- **PERIPHERAL_PROFILE_PACE_ALARM_DEFAULT_VALUE**: Default pace alarm value

Calorie Display Types

- **CALORIES_DISPLAY_TYPE_ALL**: Total calories, metabolism + exercise
- **CALORIES_DISPLAY_TYPE_ACTIVE**: Exercise calories

Notification Handlers

These methods handle the notifications received from the Cling SDK, providing feedback and updates on device states.

`bleRegisterError(_:)`

- **Purpose**: Handles errors that occur during device registration.

`bleConnectDidChange(_:)`

- **Purpose**: Listens for changes in the device's connection state and updates the app accordingly.

`bleDiscoverRefresh(_:)`

- **Purpose**: Triggered when a device is discovered during scanning.

`bleDownloadPhoneSettings(_:)`

- **Purpose**: Ensures that the device receives phone settings after connection.
- In this notification, all the original configurations are written to the device. When the device is connected or reconnected, the device needs to be reconfigured to ensure that the original configuration is valid. For example: weather configuration, reminder configuration, etc. After setting, please call downloadPhoneSettingFinished `ClingBLEModel.sharedInstance().downloadPhoneSettingFinished()` again. This method must be written, otherwise it will cause the subsequent configuration of the device to fail.

`bleSyncingData(_:)`

- **Purpose**: Updates the app on the progress of syncing data between the device and the app. notification.object = {"total":xx,"current":xx}
-

`bleReceiveMinuteData(_:)`

- **Purpose**: Receives minute-by-minute data(ClingMinuteData) from the device, such as heart rate.
- `ClingMinuteData`
- `unsigned int intTimestamp`: 时间戳 / Timestamp
- `unsigned short shortWalkStep`: 一分钟内，走路步数 / Walking steps within one minute
- `unsigned short shortRunStep`: 跑步步数 / Running steps
- `unsigned short shortDistance`: 距离 单位：米 / Distance in meters

- `unsigned short shortSleepSecond`: 睡眠时间 (手环返回的分钟数据中，不包括具体睡眠状态) / Sleep duration in seconds (No specific sleep states)
- `unsigned short shortSleepState`: 睡眠状态 (0:默认 1:醒着 2:潜睡 3:深睡) / Sleep state (0-Default, 1-Awake, 2-Light, 3-Deep)
- `float fCalories`: 卡路里 / Calories burned
- `float fSkinTemperature`: 体表温度 / Skin temperature
- `float fBodyTemperature`: 身体体温 / Body temperature
- `unsigned short shortHeartRate`: 心率 / Heart rate
- `BOOL bWear`: 是否穿戴 / Whether the device is worn
- `ClingMinuteActType acttype`: 运动类型 / Activity type
- `short shortUv`: uv指数（uv手环有效） / UV index (valid for UV wristbands)
- `int activityData`: 计算睡眠时会使用该值 / Used in sleep calculation
- `int iVoc`: voc指数 (voc手环有效) / VOC index (valid for VOC wristbands)
- `int iAlcohol`: 酒精浓度 (voc手环有效) / Alcohol concentration (valid for VOC wristbands)
- `double fSweatSugar`: 汗液糖分 / Sweat glucose level
- `double fLacticAcid`: 乳酸值 / Lactic acid level
- `double fAtmPressure`: 气压 / Atmospheric pressure
- `int spo2`: 血氧 / Blood oxygen level (SpO2)
- `float fspo2`: 血氧 (float类型) / Blood oxygen level (float type)
- `float sdnn`: 心率变异性 (SDNN) / Heart rate variability (SDNN)
- `float stress`: 压力值 / Stress level
- `float glucose`: 血糖值 / Blood glucose level
- `int bp_low`: 低压 / Diastolic blood pressure
- `int bp_high`: 高压 / Systolic blood pressure

`bleFirmwareUpdateProgress(_:)`

- **Purpose**: Handles the progress of a firmware update. notification.object = {"progress":xx} 【range 0-1】

`bleReceiveGpsData(_:)` and `bleReceiveGpsSummaryInfo(_:)`

- **Purpose**: Handles incoming GPS data and GPS summary information from the device.
- The wearable device will automatically communicate with the SDK and send the workout data to the application the first time it connects after completing the workout record, without the need for the application to actively call the function.
- Field Description:
- **ts**: 时间戳 (Timestamp in seconds)
- **si**: 秒数索引 (Second index in seconds)
- **speed**: 速度 (固件乘以100后返回的速度值) (Speed, value returned by firmware multiplied by 100)
- **lng**: 经度 (Longitude in degrees)
- **lat**: 纬度 (Latitude in degrees)

- **alt**: 海拔 (Altitude in meters)
- **distance**: 当前点与上一点的距离差 (Distance between current and previous point in meters)
- **distmax**: 最大距离 (Maximum distance in meters)
- **hr**: 心率 (Heart rate in beats per minute, bpm)
- **stamina**: 耐力 (Stamina, value less than 100)
- **aerobic**: 有氧耐力 (Aerobic stamina, value less than 100)
- **anaerobic**: 无氧耐力 (Anaerobic stamina, value less than 100)
- **kcal**: 消耗卡路里 (Calories burned in kcal)
- **kcalmax**: 最大卡路里消耗 (Maximum calories burned in kcal)
- **cadence**: 步频 (Cadence, steps per minute, range 0-255)
- **steps**: 步数 (Number of steps)
- **workouttype**: 锻炼类型 (Workout type)
- **laptime**: 圈次时间 (Lap time, in seconds)
- **strokes**: 划水次数 (Number of strokes)
- **swimtype**: 游泳类型 (Swim type)
- **avgstrokefreq**: 平均划水频率 (Average stroke frequency)
- **maxstrokefreq**: 最大划水频率 (Maximum stroke frequency)
- **swolf**: SWOLF评分 (SWOLF score)
- **avgpace**: 平均配速 (Average pace, in seconds per distance unit)
- **lapindex**: 圈次索引 (Lap index, indicating the lap number)
- **accuracy**: 精度 (Accuracy of the GPS data)

`reciveDailyTotal(_:)`

- **Purpose**: Recive daily total data from device.

- `ClingDailyData` Fields and Descriptions

- `daybeginTime`: Start time of the day (timestamp)

- `caloriesTotal`: Total calories burned for the day

- `distanceTotal`: Total distance covered during the day

- `heartRate`: Average heart rate for the day

- `rstepTotal`: Total number of running steps

- `sleepTotal`: Total sleep seconds for the day

- `wstepTotal`: Total number of walking steps

- `stepTotal`: Total steps (walking + running)

- `skinTemperature`: Average skin temperature for the day

- `wakeupTimes`: Number of times the user woke up during the night (invalid)

- `sleepEfficent`: Sleep efficiency (percentage)
- `caloriesSport`: Calories burned from sports activities
- `caloriesMetabolism`: Calories burned from metabolism
- `sportsTimeTotal`: Total time spent on sports activities
- `bWear`: Whether the wearable device is being worn (value may be invalid, use cautiously)
- `iVoc`: VOC index (valid for VOC bracelet)
- `iAlcohol`: Alcohol concentration (valid for VOC bracelet)
- `iBPTime`: Last blood pressure measurement time
- `iMeasureCount`: Number of blood pressure measurements taken
- `iBPHigh`: Last measured high blood pressure value
- `iBPLow`: Last measured low blood pressure value
- `glucose`: Blood glucose level
- `weightkg`: Body weight in kilograms
- `bodyfat`: Body fat percentage
- `iWeightTime`: Last weight measurement time
- `iWeightMeasureCount`: Number of weight measurements taken
- `sleepLight`: Duration of light sleep (in seconds)
- `sleepSound`: Duration of deep sleep
- `sleepRem`: Currently not in use
- `shortUv`: Latest UV index
- `shortActtype`: Invalid value, can be ignored
- `rssi`: Invalid value, can be ignored
- `fSweatSugar`: Sweat sugar level
- `fLacticAcid`: Lactic acid level
- `fAtmPressure`: Atmospheric pressure

- `spo2`: Blood oxygen saturation (SpO2)
- `sdnn`: Standard deviation of normal-to-normal intervals (Heart rate variability)
- `stress`: Stress level

Before obtaining and using blood pressure data, You need to call `ClingBLEModel.sendBPCMessage(intHightVal, lowPressure: intLowVal)` first to calibrate blood pressure

`reciveSosMessage()`

- **Purpose**: recive sos message from device (The specified device is valid)

`reciveFindMessage()`

- **Purpose**: recive find phone message from device

`caculateSleep(_,_)`

- **Purpose**: caculate sleep infomation
- `SleepModel` Fields and Descriptions
- `mlStartTime`: Start time of the sleep session (timestamp)
- `mlEndTime`: End time of the sleep session (timestamp)
- `mnSleepState`: Sleep state (e.g., awake, light sleep, deep sleep) (0: no value; 1: awake; 2: light sleep; 3: sound(deep) sleep; other: mid)

Methods:

- `getDuration()`: Get the duration of the sleep session
- `getMiddleTimestamp()`: Get the middle timestamp of the sleep session
- `toString()`: Convert the sleep model data to a string representation

---

- `SleepCycle` Fields and Descriptions
- `mnCycleNumber`: Sleep cycle number
- `mlStartTime`: Start time of the sleep cycle (timestamp)

- `mlEndTime`: End time of the sleep cycle (timestamp)
- `mnHeartRateLightSleep`: Average heart rate during light sleep
- `mfSkinTemperatureLightSleep`: Skin temperature during light sleep
- `mnHeartRateSoundSleep`: Average heart rate during deep sleep
- `mfSkinTemperatureSoundSleep`: Skin temperature during deep sleep
- `mfBodyTemperatureLightSleep`: Body temperature during light sleep
- `mfBodyTemperatureSoundSleep`: Body temperature during deep sleep

Methods:

- `getMiddleTimestamp()`: Get the middle timestamp of the sleep cycle
- `toString()`: Convert the sleep cycle data to a string representation

---

- `SleepPercentage` Fields and Descriptions
- `iAwakePt`: Percentage of time spent awake
- `iLightPt`: Percentage of time spent in light sleep
- `iMiddlePt`: Percentage of time spent in intermediate sleep (if applicable)
- `iSoundPt`: Percentage of time spent in deep sleep
- `SleepUtil` Methods and Descriptions
- `getClassfiedSleepState(arrlstMin)`: Classifies sleep states based on minute-level data. Returns an array representing classified sleep states.
- `getClassfiedSleepCycle(arrlstSleep, nAge)`: Classifies sleep cycles based on sleep data and age. Returns an array representing classified sleep cycles.
- `calculateSportParamsAvg(arrlstCycle, arrlstMin)`: Calculates the average sports-related parameters for each sleep cycle.
- `calculateSportParams(arrlstCycle, arrlstMin)`: Calculates sports-related parameters for each sleep cycle.
- `isBestAwakePeriod(nCycleIndex, nAge)`: Determines whether the given sleep cycle is the best period to wake up based on age. Returns a boolean.

- `getSleepScore(arrlstData, arrlstCycle, level, gender, age, sleepSeconds, deepSeconds)`: Calculates the sleep score based on various sleep parameters. Returns an integer representing the sleep score.

- `getSleepPercent(arrlstSleep)`: Calculates the percentage distribution of different sleep stages. Returns a `SleepPercentage` object.

- `getStartAndEndSleep(array)`: Extracts the start and end sleep times from classified sleep states. Returns an array where the first object is the sleep start time and the last object is the wake-up time.

ClingUtilsModel Properties and Methods

Properties

- `iStepGoal`: Step goal (updated based on health index retrieval).
- `iSleepGoal`: Sleep goal.
- `iCalorieGoal`: Calorie goal for exercise.

Methods

- `sharedInstance()`: Singleton instance of ClingUtilsModel.

- `getStepsPointWithLevel(healthLevel, gender, age, steps)`: Computes step score.

- `getSleepPointWithLevel(healthLevel, gender, age, sleepMinutes)`: Computes sleep score.

- `getCalsPointWithLevel(healthLevel, gender, age, calsSport)`: Computes calorie score.

- `getHeartratePointWithLevel(healthLevel, gender, age, model)`: Computes heart rate score.

- `getRestHeartratePointWithLevel(healthLevel, gender, age, restHr)`: Computes resting heart rate score.

- `getSportHeartratePointWithLevel(healthLevel, gender, age, model)`: Computes exercise heart rate score.

- `getTempPointWithModel(model, hr)`: Computes body temperature score.

- `socreTotalLev(lv, heartRate, temperature, sleep, calories, steps, bloodPre)`: Computes the overall health index.

- `setGenerateTimelineWalkBubble(walk, run)`: Sets time threshold for generating walking and running bubbles.

- `generateSportBubble(array)`: Generates sport bubbles from minute data.

- `mergeBubble(array)`: Merges consecutive bubbles of the same activity.

- `calculateDayTotalData(array, daybegin)`: Computes daily total data from minute data.
- `calculateHealthEvalutionScore(pageSelection, gender)`: Computes health evaluation score.
- `getDayGoal(healthLevel, gender, age, type)`: Computes daily activity goals.
- `getSleepInterval()`: Retrieves sleep bubble generation interval.
- `getActivityInterval()`: Retrieves activity bubble generation interval.
- `getNormalInterval()`: Retrieves default bubble generation interval.
- `checkSleepState(minuteDatas, isHR)`: Identifies sleep states from minute data. `minuteDatas` Minute-level data for an entire day (12:00 PM to 11:59 AM of the next day). If the data is incomplete, it covers 12:00 PM to the current time. `isHR` Indicates whether the current device supports heart rate monitoring.

Helper Methods

`reloadState(_:)`

- **Purpose**: Updates the UI to reflect the current device state (connected or disconnected).

This documentation provides an overview and detailed explanation of the `ViewController` class, focusing on its workflow and key methods.